ORM

What is an ORM?

ORM = Object Relational Mapper

It lets you talk to a **database using code objects instead of SQL**.

Instead of writing SQL like this:

```
SELECT * FROM users WHERE id = 1;
```

You write something like:

```
const user = await prisma.user.findUnique({
  where: { id: 1 }
});
```

So:

Database	Code
Table	Model
Row	Object
Column	Field

users
---
id
name
email

Prisma model

```
model User {
  id    Int     @id @default(autoincrement())
  name  String
  email String
}
```

Now Prisma converts your code → SQL automatically.

Think of ORM like a **translator between your backend code and the database**.

Node.js code
↓
Prisma
↓
PostgreSQL

2. Why ORMs Exist

Without ORM:

You manually write SQL everywhere.

Problems:

• SQL scattered across files
• Harder refactoring
• Harder relations
• More boilerplate

ORM gives:

✓ Type safety
✓ Cleaner code
✓ Schema management
✓ Relations handling
✓ Migrations

Prisma Architecture (Important)

Prisma has 3 main parts.

1️⃣ Prisma Schema

Defines database structure.

schema.prisma

```
model User {
 id    Int @id @default(autoincrement())
 name  String
 email String
}
```

2️⃣ Prisma Client

Auto-generated JS client.

Used in code:

```
const users = await prisma.user.findMany()
```

3️⃣ Prisma Migrate

Handles database migrations.

Example:

```
npx prisma migrate dev
```

Creates SQL migration and updates database.

let's integrate Prisma with PostgreSQL

Step 1 — Install dependencies

Inside your backend project:

```
npm install prisma --save-dev
npm install @prisma/client
```

Explanation:

Package	Purpose
prisma	CLI tool
@prisma/client	Query database

Step 2 — Initialize Prisma

Run:

```
npx prisma init
```

This creates:

```
prisma/
  schema.prisma

.env
```

Step 3 — Configure PostgreSQL connection

Inside .env

Example:

```
DATABASE_URL="postgresql://postgres:password@localhost:5432/mydb"
```

Structure:

```
postgresql://USER:PASSWORD@HOST:PORT/DATABASE
```

Example with docker:

postgresql://postgres:postgres@localhost:5432/postgres

Step 4 — Configure Prisma provider

Open:

prisma/schema.prisma

Edit:

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

Step 5 — Create your first model

Example:

```
model User {
  id    Int     @id @default(autoincrement())
  name  String
}
```

Step 6 — Run Migration

Run:

```
npx prisma migrate dev --name init
```

What happens:

Prisma schema
↓
Migration SQL generated
↓
Database tables created

Now PostgreSQL gets:

users table

Step 7 — Generate Prisma Client

Usually automatic, but manually:

```
npx prisma generate
```

Step 8 — Use Prisma in Node.js

Example:

```
const { PrismaClient } = require("@prisma/client");

const prisma = new PrismaClient();
```

Example: Create User

```
const user = await prisma.user.create({
  data: {
    name: "Prajwal"
  }
});

console.log(user);
```

Example: Get Users

```
const users = await prisma.user.findMany();

console.log(users);
```

Example: Get One User

```
const user = await prisma.user.findUnique({
  where: { id: 1 }
});
```

Example: Delete User

```
await prisma.user.delete({
  where: { id: 1 }
});
```

The Prisma Change Cycle (Mental Model)

Whenever you **add a table / change a column / add relation**, you repeat this loop:

Edit Schema
↓
Run Migration
↓
Generate Client

That’s it.

1️⃣ Prisma Relations

This is how tables connect.

Database thinking:

User
↓
Posts
↓
Comments

Prisma Schema Example

```
model User {
 id    Int    @id @default(autoincrement())
 name  String

 posts Post[]
}

model Post {
 id     Int   @id @default(autoincrement())
 title   String

 authorId Int
 author   User  @relation(fields: [authorId], references: [id])
}
```

Creating Related Data

Example: create **user + post**

```
await prisma.post.create({
 data: {
  title: "My First Post",
  author: {
   connect: { id: 1 }
  }
```

```
  }
})
```

Meaning:

create post
attach to user id=1

2️⃣ Select vs Include (VERY IMPORTANT)

This confuses many developers.

include → fetch relations

Example:

```
const users = await prisma.user.findMany({
  include: {
    posts: true
  }
})
```

Result:

```
User
  ├─ name
  ├─ id
  └─ posts[]
```

Example output:

```
{
 id:1
 name:"Prajwal"
 posts:[
   { id:1, title:"Hello" }
 ]
}
```

select → choose fields

Example:

```
const users = await prisma.user.findMany({
  select: {
    name: true
  }
})
```

Result:

```
[{ name: "Prajwal" }]
```

Interview Cheat

select → choose columns
include → fetch relations

3️⃣ Transactions

Transactions ensure **all queries succeed or none run**.

Example:

Transfer money

deduct from A
add to B

If one fails → rollback.

Prisma Transaction

```
await prisma.$transaction([
  prisma.user.update({
    where: { id: 1 },
    data: { balance: { decrement: 100 } }
  }),

prisma.user.update({
where: { id: 2 },
data: { balance: { increment: 100 } }
```

```
})
])
```

4️⃣ Pagination (Very Common in APIs)

Used when you have **1000+ rows**.

Example API:

GET /posts?page=2

Offset Pagination

```
const posts = await prisma.post.findMany({
  skip: 10,
  take: 10
})
```

Meaning:

skip first 10
take next 10

SQL equivalent:

LIMIT 10 OFFSET 10

Cursor Pagination (advanced)

Better for large datasets.

```
const posts = await prisma.post.findMany({
  take: 10,
  cursor: { id: 20 }
})
```

Meaning:

start after id 20
fetch next 10

5️⃣ Filtering

Filtering = SQL WHERE.

Example:

GET /posts?author=1

Prisma:

```
const posts = await prisma.post.findMany({
  where: {
    authorId: 1
  }
})
```

Advanced Filtering

```
const posts = await prisma.post.findMany({
  where: {
    title: {
      contains: "Prisma"
    }
  }
})
```

Result:

posts containing "Prisma"

Multiple Filters

```
where: {
  authorId: 1,
  title: {
    contains: "API"
  }
}
```

SQL equivalent:

```
WHERE authorId = 1
AND title LIKE '%API%'
```